ENGR -222: Intelligent Systems II **Week 5 Lab**

The goals of this lab are

1. Create relevant endpoints for individualized servers using Swagger Editor
2. Create functions to complement the endpoints
3. Test the service endpoints using the virtual machines on Jetstream2

Part I. Github and Jetstream2

1. Log into your ACCESS account and pull up your Jetstream2 instance (your virtual machine).
2. Navigate to your Documents/work directory where you cloned the E222 Student repository and your individual repository.
3. Navigate to your individual Git repository and perform a git pull to update contents.
4. Navigate to the E222 Student repository and perform a git pull to update contents.
5. Copy Week 5 contents from E222 Student to your individual repository.
6. Navigate to your individual repository/Week 5 folder. You should find a folder called myFirstApp, similar to what we had last time.
7. Activate your virtual environment: `source .venv/bin/activate`

In pairs, answer these questions:
What is in the src folder (in general)?

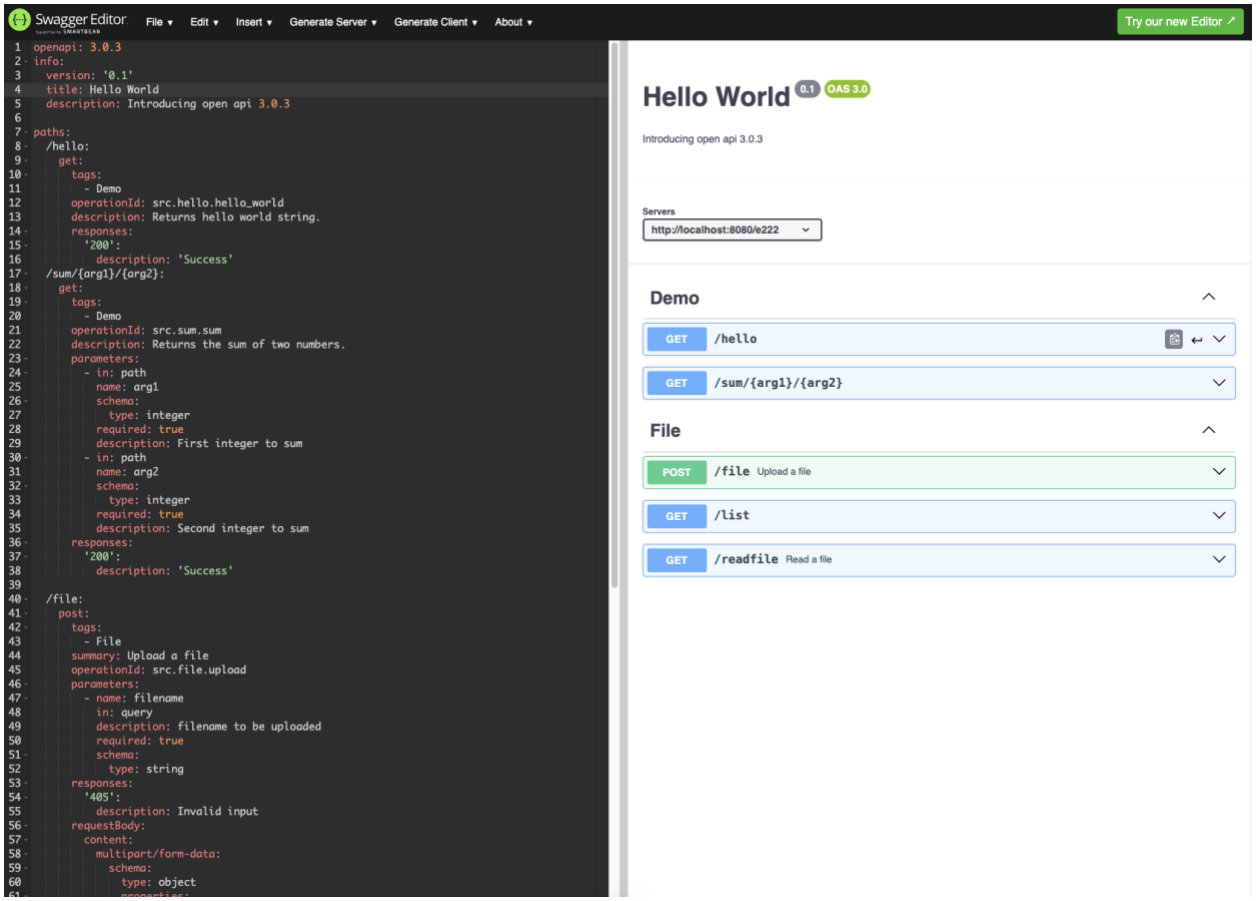
What is in the requirements.txt document (use VIM to inspect)?

What is a virtual environment?

Part II. Endpoint Exploration

The Swagger Editor is a tool for us to create our OpenAPI Specification file. This file defines and documents our application interface while complying with REST infrastructure. The Specification file will be exported as a YAML file.

1. Navigate to the Swagger Editor and let's create this file for our service. Copy and paste the new master.yml file from the E222 service into this editor. It should look similar to what is below:



2. Change the Title to something more meaningful to your prediction service.
3. Explore the endpoints (we will discuss these things as a group). Make sure you can identify:
    a. An endpoint with path parameters. How do you indicate this in the request definition?
    b. An endpoint with query parameters. How dyou indicate this in the request definition?
    c. A POST vs. a GET request. What do you notice about the definitions?
    d. Try one out – does it work? Why or why not?

4. Let's spin up our service on our VM. If you changed your title in the Swagger Editor, you will need to update the master.yml file in your VM terminal to reflect the change.
    a. Type `python3 server.py`
    b. Open another tab and type in `http://YOUR.IP.ADDRESS.X/e222/ui`
    c. This should be the same as the Swagger Editor.
        i. Test out the **/hello** endpoint
        ii. Explore the **/sum** endpoint (you may want to use another tab to execute the http request if provides you!

iii. Test out the /file endpoint. What does it do?
iv. Test out the /list endpoint. What does it do?
v. What about /readfile?

Terminate your service (CTL+C).

Part III. YOUR Endpoints

In this section, you will work to create your own endpoints for your service. The goals is to have two – three completed today. Remember that each endpoint needs:

- Properly defined requests in the yml file
- Python scripts to execute the required function
- (Sometimes you need to install more libraries, depending on the scripts you right. If you have to install something new, update the requirements.txt file!)

1. /model_info endpoint
    a. Create a /model_info endpoint in your master.yml file. Use the appropriate syntax. I recommend you doing this in your Swagger Editor and copy/paste your VM master.yml file when complete.
    b. You will note there is a model.py file in your src directory. It is UNFINISHED. Complete it with your model specifics.
    c. Test your service and your endpoint.
2. /prediction endpoint
    a. Create a /prediction endpoint in your master.yml file. This file should be a POST that allows someone to upload a file to the server (like testdata.csv) and return a list of resulting predictions. Use the appropriate syntax. I recommend you doing this in your Swagger Editor and copy/paste your VM master.yml file when complete.
    b. Note there a file_predict function in model.py. It is not finished as it needs YOUR model to be loaded. This means you need to make sure your pkl file is in the same directory (**src** folder) as this script). Make those updates.
    c. Test your service. Make sure that any installations are updated in your requirements.txt file.

src > model.py

```python
rom sklearn import datasets
from joblib import load
import numpy as np
import json

from flask import request, jsonify
import pandas as pd
UPLOAD_FOLDER='.'

#This will return a string with information about this model
def model_info():
    return 'BLAH'

#This will allow a file to be uploaded (it is used in file_predict)
def upload(filename):
    f = request.files['file']
    f.save(filename)
    return filename

#This function will load a model in the app directory, read in the cs
#test data provided by the upload and return a jsonified list of
#predictions from the test data set given.
def file_predict(filename):
    my_model = load('logs_heart_mdl.pkl')
    name = upload(filename)
    test_data = pd.read_csv(name, index_col =[0])
    test_np = test_data.to_numpy()
    pred = my_model.predict(test_np)
    pred_list = pred.tolist()
    json_str = json.dumps(pred_list)
    return json_str

```

3. /performance endpoint
    a. Create a /performance endpoint that provides accuracy or whatever performance metric you used to the user.
    b. No function has been created to do this. You will need to either create a new py file OR include it in the model.py script (recommended).

At the end of lab, make sure you:

1. **git add .**
2. **git commit -m "Week 5 labwork"**
3. **git push**
4. **deactivate your venv**
5. **SHELVE your VM instance.**

If you did not complete these endpoints, they need to be completed before next lab!